PROJECT BUILD LOG

Azure App Service Cloud Application

Build Log — PaaS Deployment, Incident Response, and Containerization Progress

Phase 1 Complete (incident resolved) · Phase 2 In Progress (Registry, Dockerfile & Identity done; container deployment pending)

Prepared by: Parikshit Arer
Resource Group: Project-2
Build Date: October 6, 2026

Table of Contents

A screenshot-verified account of the October 6 build session, including a real deployment incident and its resolution.

1. Project Overview

2. Phase 1 — PaaS Deployment — Flask on App Service

Steps 1.1, 1.2, 1.3

3. Phase 2 — Incident — Diagnosing and Resolving a 503 Error

Steps 2.1, 2.2, 2.3

4. Phase 3 — Containerization Progress

Steps 3.1, 3.2, 3.3

5. Phase 4 — Monitoring

Steps 4.1

6. Roadmap Status Update

7. Resource Inventory

8. Recommendations & Next Steps

9. Conclusion

1. Project Overview

This document records the actual build session for Project-2 on October 6, 2026: standing up a Python/Flask app on Azure App Service, diagnosing and fixing a real deployment failure, and beginning the move toward a containerized deployment backed by Azure Container Registry and a Managed Identity. It replaces the earlier architecture-only draft of this project with a screenshot-verified account of what was actually done, including the parts that didn't work on the first try.

Note on the earlier planning document: This build uses a simpler Flask page than the one described in the project's original planning notes (no APP_ENV / PROJECT_NAME environment variables are read in the deployed code below), and Application Insights was not enabled at creation time — both differences are visible directly in the screenshots that follow. This document reflects what was actually deployed in this session rather than the earlier plan.

Resource Group	Project-2	**Build Date**	October 6, 2026
Prepared by	Parikshit Arer	**Region**	Central India (see Step 1.1 note)

2. Phase 1 — PaaS Deployment — Flask on App Service

PHASE 1

PaaS Deployment — Flask on App Service

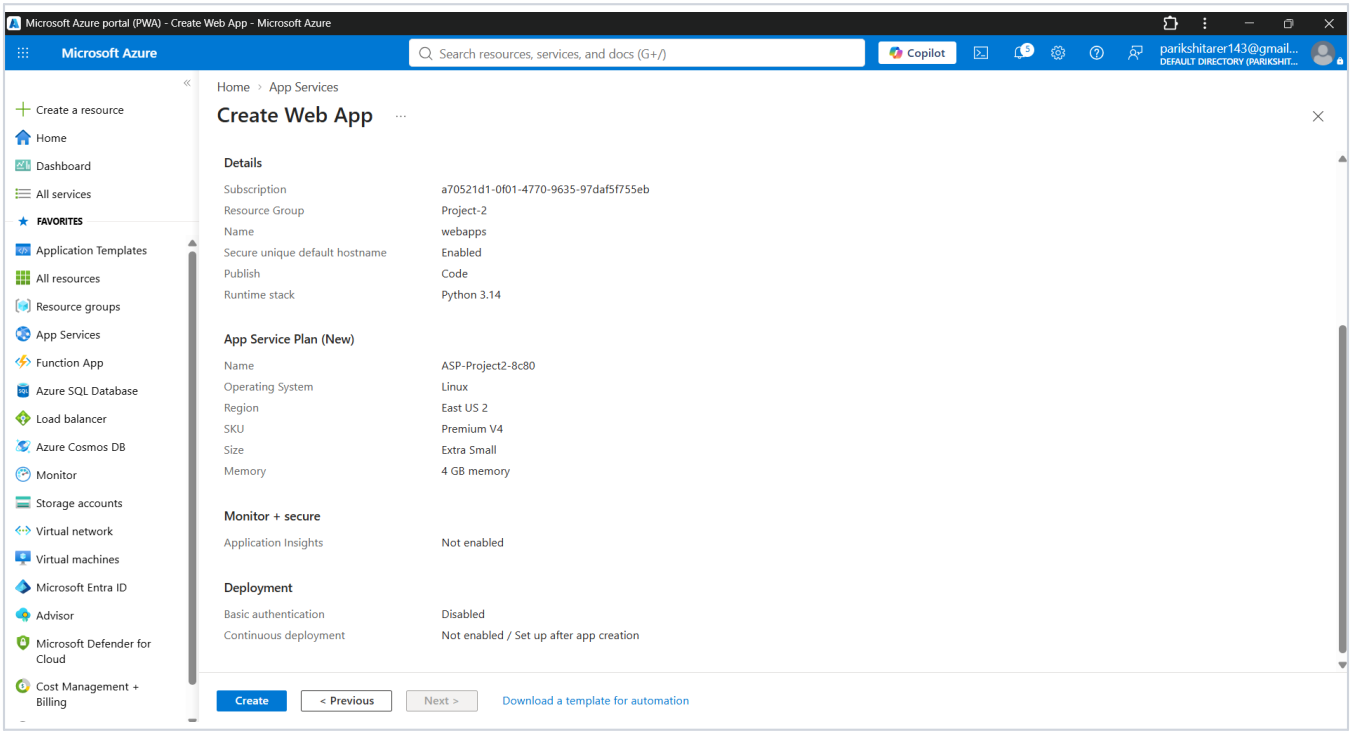
Provision the Web App and its plan, deploy the Flask application via Local Git, and set the runtime stack and startup command.

1.1 Create the Web App

Name	webapps
Resource Group	Project-2
Runtime stack	Python 3.14
Publish	Code
App Service Plan	ASP-Project2-8c80 — Linux, Premium V4, Extra Small, 4 GB
Application Insights	Not enabled (at creation)
Basic authentication	Disabled

The Web App **webapps** is created in the **Project-2** resource group on a new **Premium V4** Linux plan, targeting the **Python 3.14** runtime, deployed by **Code** rather than a container — containerization comes later, in Phase 3.

Two things worth flagging for later: Application Insights is **not** enabled at creation time here (it gets added afterward — see Phase 4), and this review screen lists the plan's region as **East US 2** — yet every hostname in the screenshots that follow resolves to **centralindia-01.azurewebsites.net**, matching the Container Registry created later in the same resource group (explicitly Central India). See Recommendations.



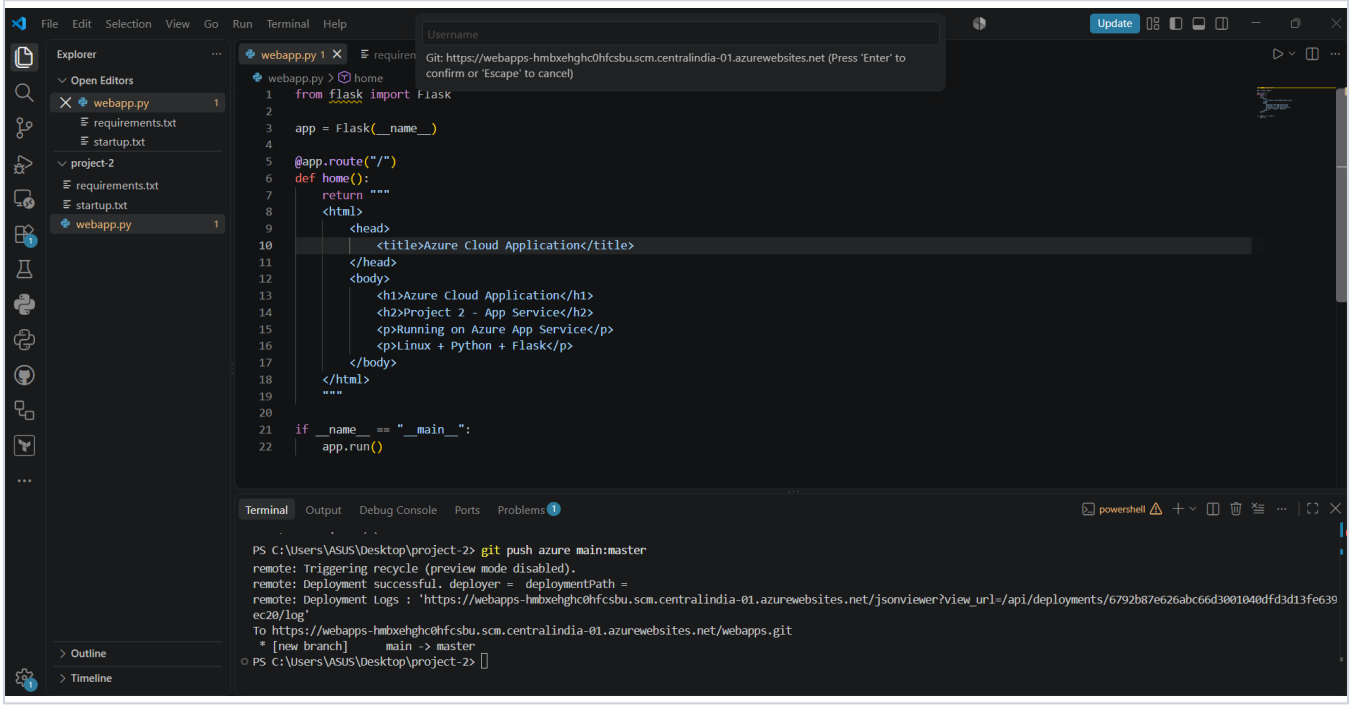
Create Web App — Review screen

1.2 Deploy the Flask Application via Local Git

File	webapp.py
Route	"/" → returns a static HTML page
Page content	“Azure Cloud Application / Project 2 - Azure App Service / Running on Azure App Service / Linux + Python + Flask”
Deploy command	git push azure main:master
Result	Deployment successful

The Flask application is a single file, **webapp.py**, with one route that returns a static HTML page identifying itself as the Project 2 Azure App Service build.

Deployment uses Azure's **Local Git** mechanism: git push azure main:master pushes straight to the Kudu build service, which reports **“Deployment successful”** and publishes to webapps-hmbxehghc0hfcsbu.scm.centralindia-01.azurewebsites.net.



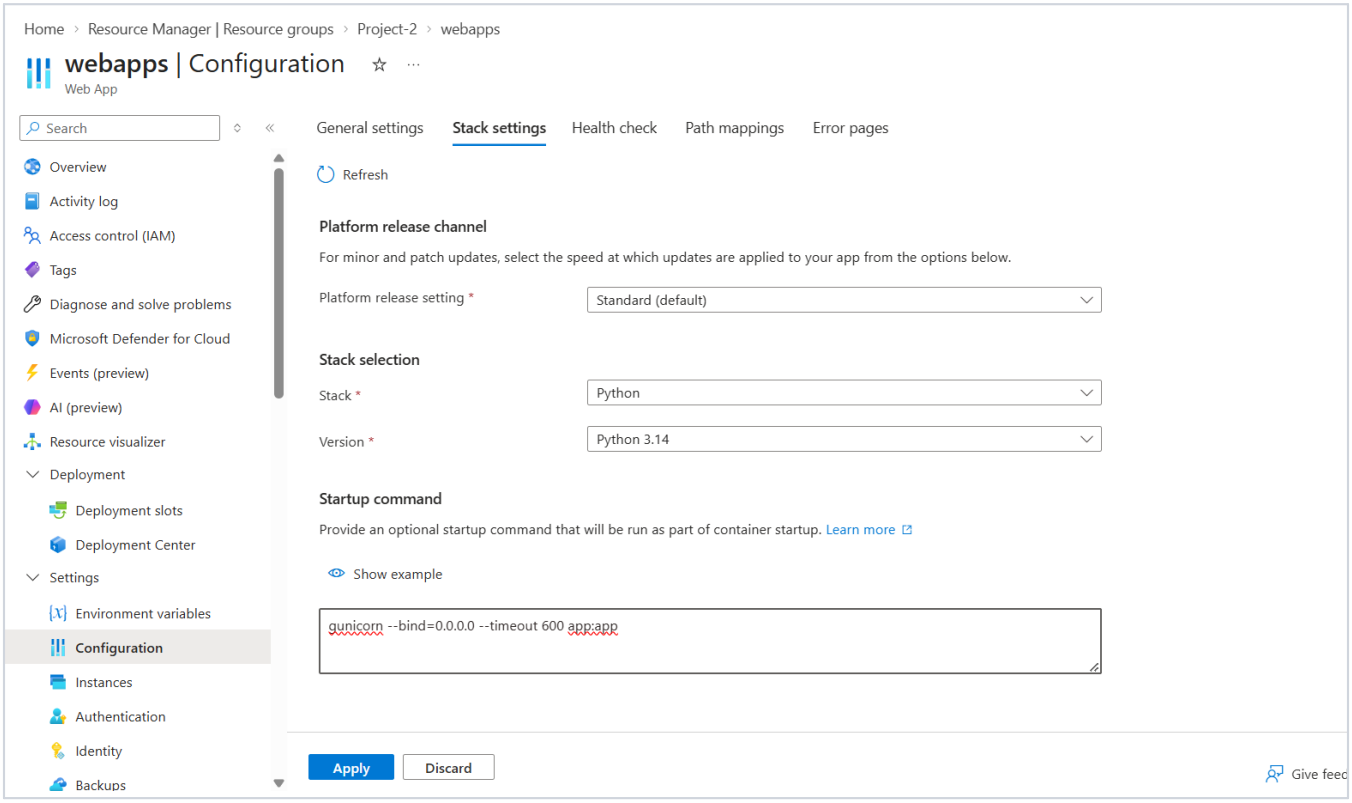
VS Code — webapp.py and git push azure main:master

1.3 Configure the Runtime Stack & Startup Command

Stack	Python
Version	Python 3.14
Startup command	gunicorn --bind=0.0.0.0 --timeout 600 app:app

The Stack settings confirm Python 3.14 and set a Gunicorn startup command targeting the module **app:app**.

That module name is worth noting now: the file deployed in Step 1.2 is **webapp.py**, not **app.py** — Gunicorn's app:app target won't find a matching module. This mismatch is the root cause diagnosed in Phase 2.



webapps | Configuration — Stack settings (initial)

3. Phase 2 — Incident — Diagnosing and Resolving a 503 Error

INCIDENT — PHASE 2

Incident — Diagnosing and Resolving a 503 Error

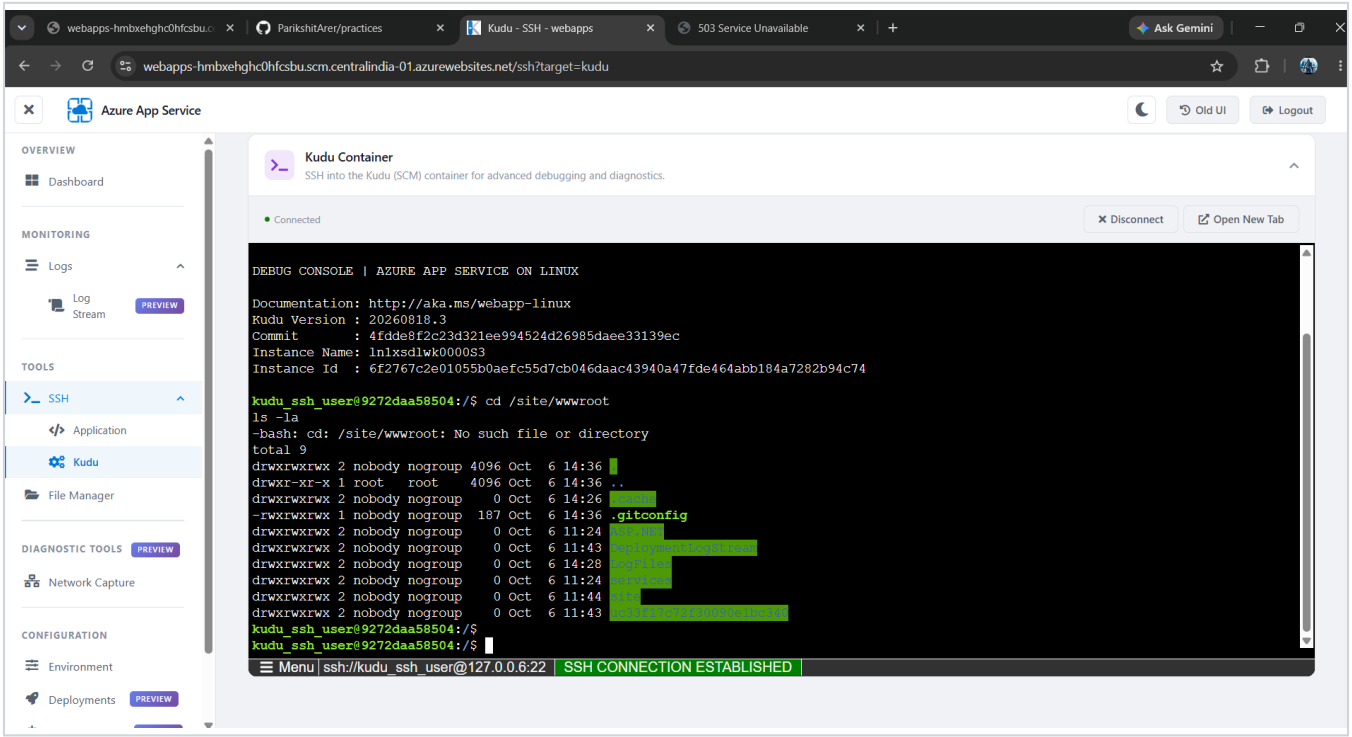
The deployment above did not serve correctly on first load. This phase documents the diagnosis, fix, and confirmation — a real incident, not a scripted success.

2.1 Diagnose via the Kudu SSH Console

Tool	Kudu Container SSH (Advanced Tools)
Command run	cd /site/wwwroot
Result	-bash: cd: /site/wwwroot: No such file or directory

With the app not responding correctly, the Kudu SSH console is opened directly against the running container for hands-on debugging — no redeploy required to look around.

The specific path tried here, /site/wwwroot, doesn't exist from the container root (the correct absolute path on Azure App Service Linux is /home/site/wwwroot), so this particular command was a dead end rather than proof the deployment itself was missing. What it does confirm is that direct container access was available for deeper diagnosis.



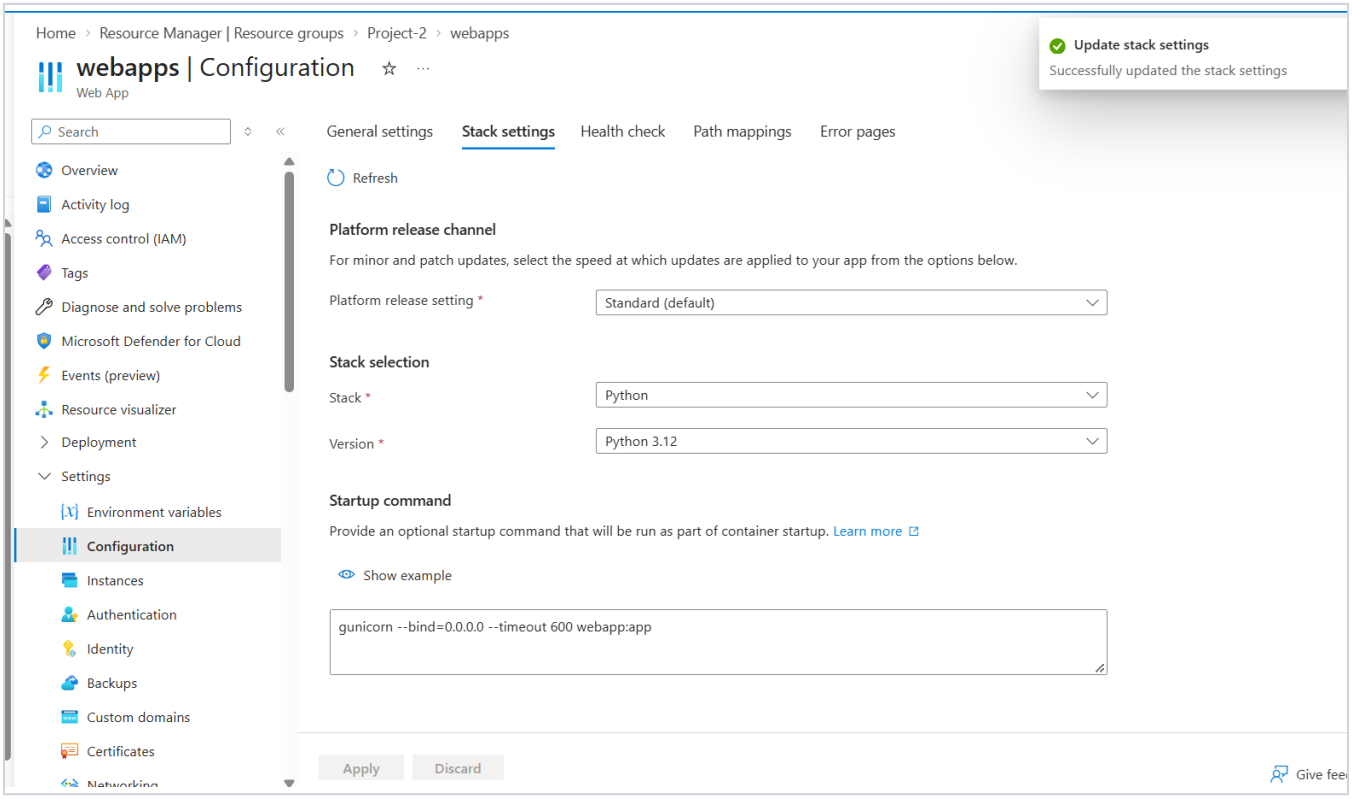
Kudu SSH console — inspecting the container filesystem

2.2 Apply the Fix — Correct Runtime Version & Startup Command

Version changed	Python 3.14 → Python 3.12
Startup command changed	gunicorn --bind=0.0.0.0 --timeout 600 app:app → gunicorn --bind=0.0.0.0 --timeout 600 webapp:app
Result	“Update stack settings — Successfully updated the stack settings”

Two changes land together here: the Python version steps back from **3.14** (a very new runtime) to the more established **3.12**, and — the change with direct evidence tying it to the root cause — the Gunicorn target is corrected from **app:app** to **webapp:app**, matching the actual file name from Step 1.2.

Either change alone might have mattered; together, they're what the next step confirms fixed the deployment.



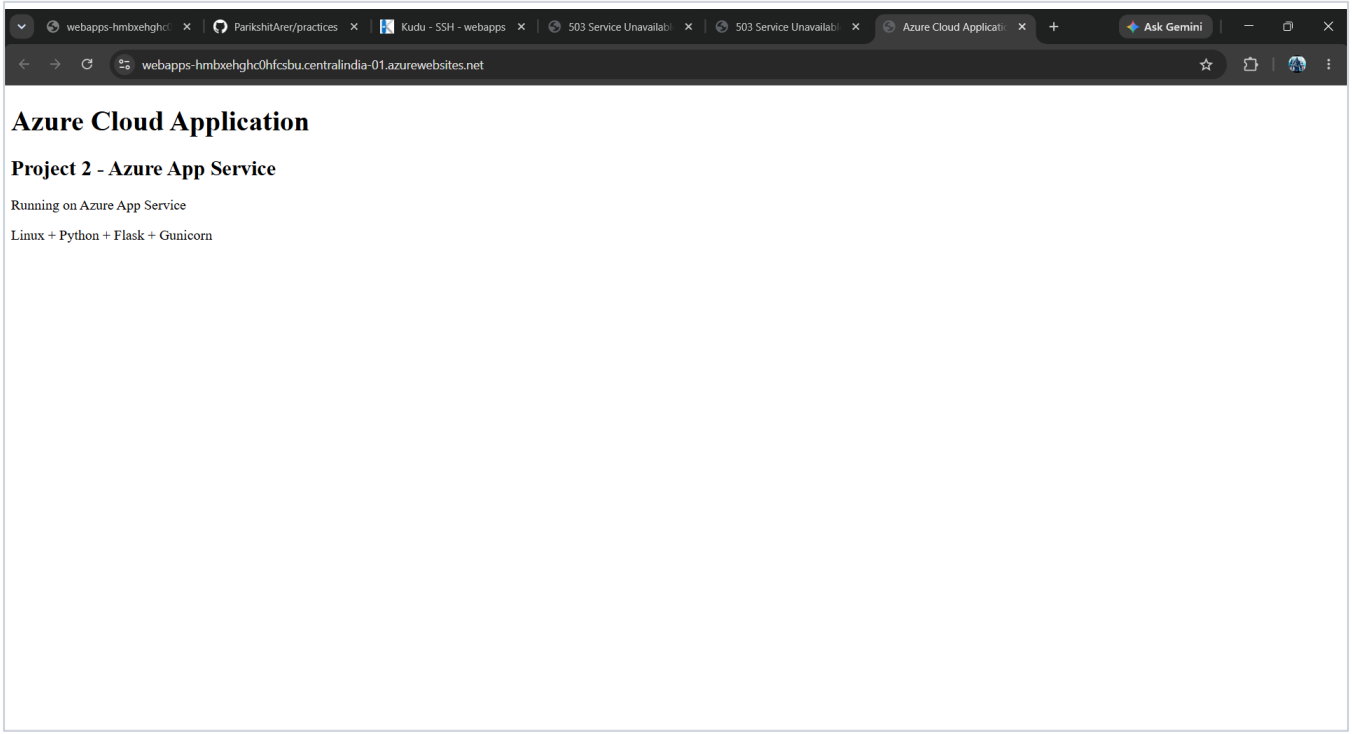
webapps | Configuration — Stack settings updated, “Successfully updated”

2.3 Confirm Resolution — Application Loads Successfully

URL	webapps-hmbxehghc0hfcsbu.centralindia-01.azurewebsites.net
Result	Page renders: Azure Cloud Application / Project 2 - Azure App Service

The corrected configuration takes effect and the app now serves its page correctly. The browser's own tab history tells the rest of the story — two tabs still open read “503 Service Unavailable” from earlier attempts, alongside the Kudu SSH tab used for diagnosis in Step 2.1.

This is the moment the incident closes: symptom (503), diagnosis (Kudu SSH + configuration review), fix (stack version and startup command), and confirmation (a correctly rendered page), all captured in sequence.



Browser — the application now renders correctly

4. Phase 3 — Containerization Progress

PHASE 3

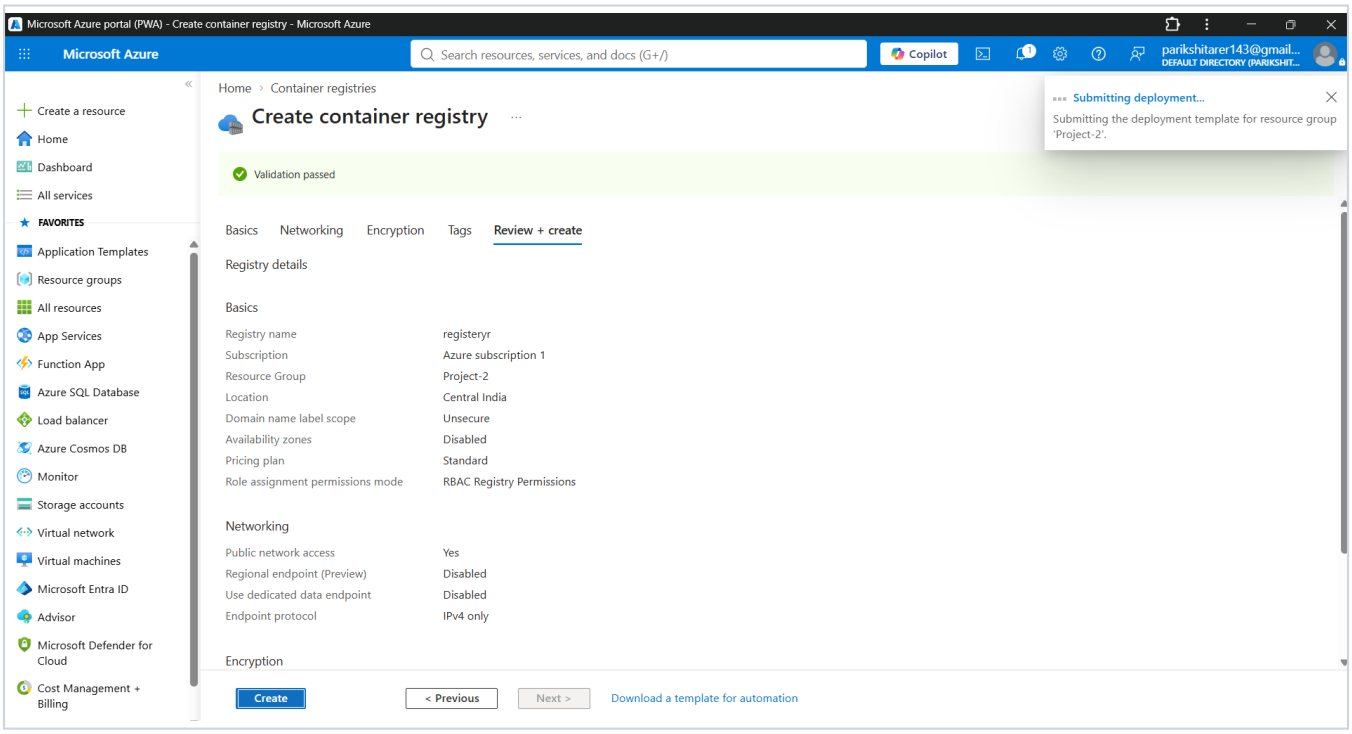
Containerization Progress

With Phase 1 stable, work begins on Phase 2 of the roadmap: a private registry, a Dockerfile, and a Managed Identity to pull images without stored credentials.

3.1 Create Azure Container Registry

Registry name	registeryr
Resource Group	Project-2
Location	Central India
Pricing plan	Standard
Role assignment permissions mode	RBAC Registry Permissions
Public network access	Yes

A new Azure Container Registry, **registeryr**, is provisioned in the same resource group and region as the rest of the project. **RBAC Registry Permissions** mode is selected rather than the legacy admin-account model — the right choice for the Managed Identity approach planned in Step 3.3.



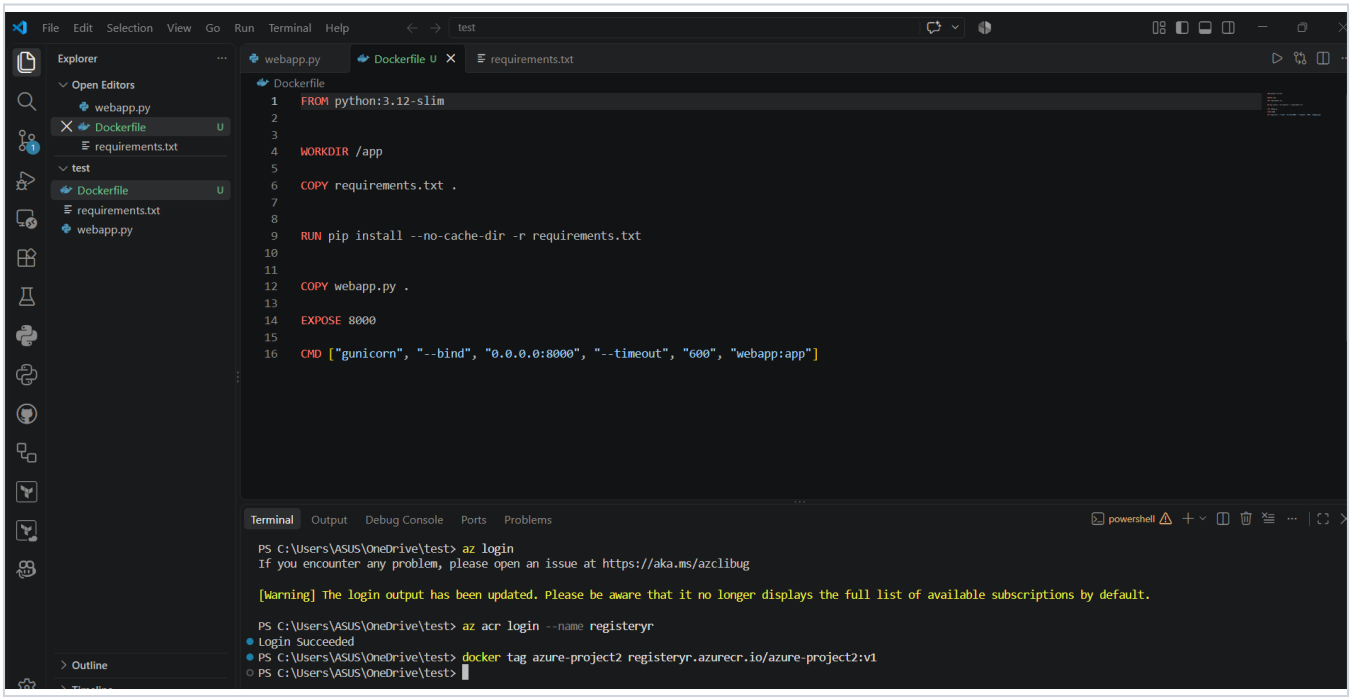
Create container registry — Review + create

3.2 Author the Dockerfile & Tag the Image

Base image	python:3.12-slim
Workdir	/app
Installs	requirements.txt via pip
Copies	webapp.py
Exposes	8000
CMD	gunicorn --bind 0.0.0.0:8000 --timeout 600 webapp:app
Terminal	az acr login --name registeryr → Login Succeeded; docker tag azure-project2 registeryr.azurecr.io/azure-project2:v1

The Dockerfile mirrors the fix from Phase 2 exactly — its CMD targets **webapp:app**, matching the corrected startup command, and uses the same **Python 3.12** line that resolved the incident. That continuity is a good sign: the lesson from the 503 carried forward into the container build rather than reintroducing the same mismatch.

In the terminal, az acr login --name registeryr succeeds, and the local image azure-project2 is tagged as registeryr.azurecr.io/azure-project2:v1 — ready for docker push, which hadn't run yet as of this screenshot.



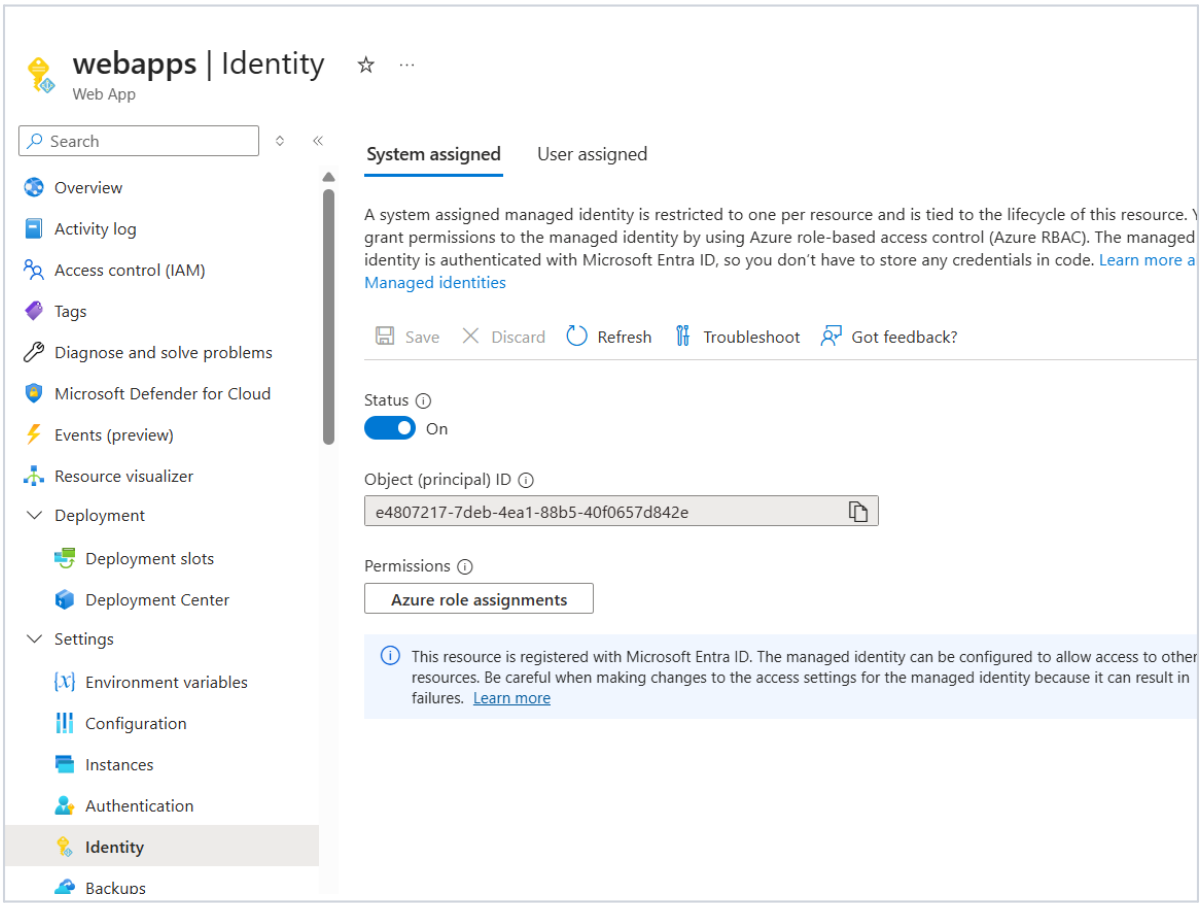
VS Code — Dockerfile, plus az acr login and docker tag in the terminal

3.3 Enable Managed Identity for Secure ACR Access

Identity type	System assigned
Status	On
Object (principal) ID	e4807217-7deb-4ea1-88b5-40f0657d842e
Permissions	Azure role assignments — not yet opened/confirmed

A system-assigned Managed Identity is switched on for **webapps**, giving it an identity in Microsoft Entra ID independent of any stored credentials.

The next action — granting this identity the **AcrPull** role on **registeryr** — is the piece that actually lets App Service pull the image without a username or password. That role assignment hadn't been made as of this screenshot; see Recommendations.



webapps | Identity — System assigned identity turned On

5. Phase 4 — Monitoring

PHASE 4

Monitoring

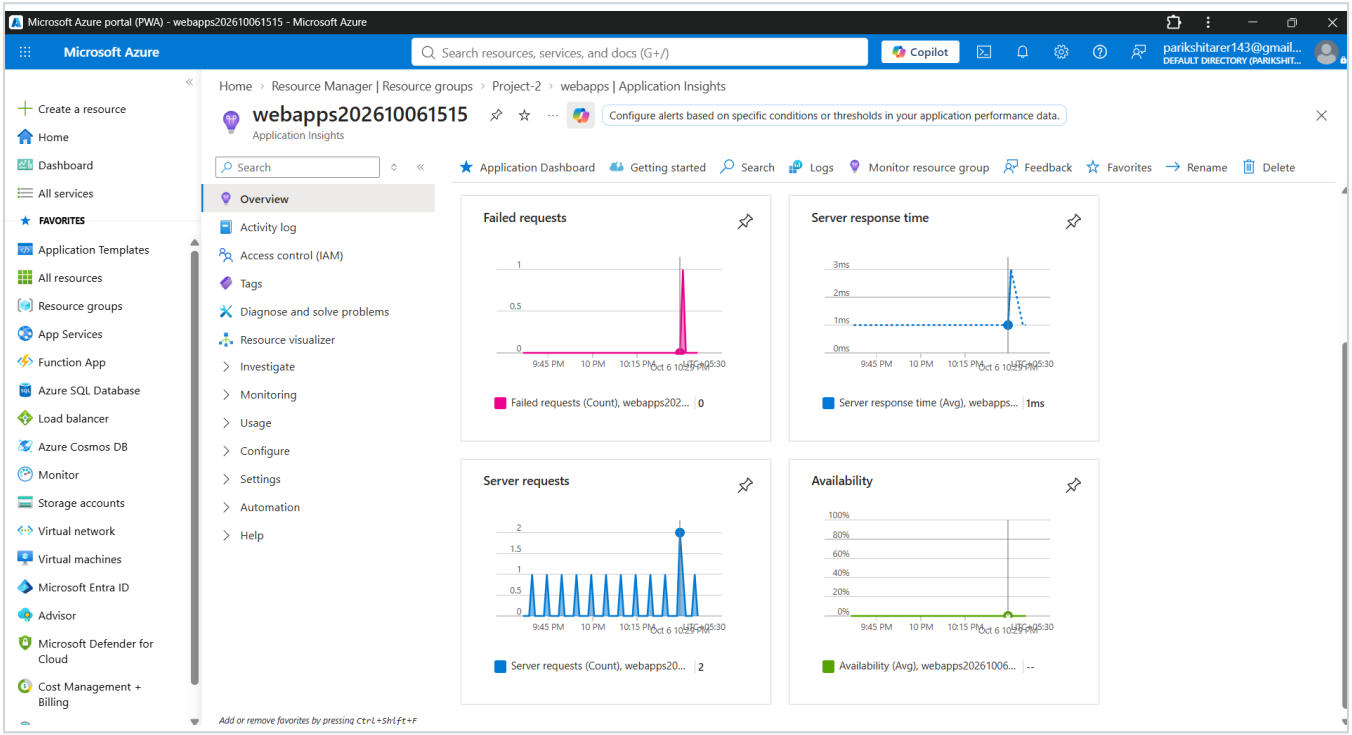
A check-in on Application Insights telemetry for the current (code-based) deployment.

4.1 Review Application Insights Telemetry

Resource	webapps202610061515
Failed requests	A single brief spike to 1, otherwise 0 across the window
Server response time	~1ms average, with one spike toward 3ms late in the window
Server requests	A steady periodic pattern (~1 per interval), with one spike to 2
Availability	Legend shows "—" — not enough data yet for an average

With Application Insights now active, the Overview dashboard shows a healthy pattern: requests arriving at a steady, low rate (consistent with automated pings rather than real traffic), response times around 1ms, and only one brief failed-request blip across the monitored window.

The Availability tile hasn't accumulated enough data to show a percentage yet — worth revisiting once the app has been running, and ideally an Availability test has been configured, for longer.



Application Insights — Overview dashboard

6. Roadmap Status Update

Cross-referencing this session's evidence against the 10-item Phase 2 roadmap from the project's planning document.

#	Step	Status	Evidence
1	**Dockerize the Flask application**	DONE	Dockerfile authored: python:3.12-slim base, installs requirements.txt, copies webapp.py, exposes 8000, runs gunicorn webapp:app.
2	**Build and test the Docker image locally**	PARTIAL	An image (azure-project2) exists and was tagged for the registry; no screenshot captures a local docker run / test pass.
3	**Create Azure Container Registry (ACR)**	DONE	registeryr created in Project-2, Central India, Standard tier, RBAC registry permissions.
4	**Push the Docker image to ACR**	NOT YET	az acr login succeeded and the image was tagged (registeryr.azurecr.io/azure-project2:v1); no docker push has been captured yet.
5	**Configure App Service to run the container**	NOT YET	webapps is still running as a Code (Local Git) deployment, not a Container deployment.
6	**Add a Managed Identity for secure ACR access**	PARTIAL	System-assigned identity is enabled on webapps; the AcrPull role assignment on registeryr has not yet been confirmed.
7	**Test the containerized application**	NOT YET	Depends on steps 4–6 above being completed first.
8	**Final monitoring and testing pass**	PARTIAL	Application Insights is active and was reviewed for the current code-based deployment; the containerized version has not yet been monitored.
9	**Produce architecture diagrams and a GitHub README**	PARTIAL	Architecture diagrams were produced in the prior planning document; this build log is the evidence base a README can now be written from.
10	**Add the project to a resume / portfolio**	NOT YET	Recommended once containerized deployment (steps 4–7) is complete.

7. Resource Inventory

Resources confirmed by direct screenshot evidence across this build session.

Resource Name	Type	Role in Architecture
Project-2	Resource group	Container for every resource below
webapps	App Service (Web App)	Hosts the Flask application, Linux, Python runtime
ASP-Project2-8c80	App Service Plan	Premium V4, Extra Small, 4 GB — compute backing the Web App
webapps202610061515	Application Insights	Telemetry and monitoring for the webapps Web App
registeryr	Azure Container Registry	Standard tier — will host the Docker image for Phase 2

8. Recommendations & Next Steps

Concrete next actions, grounded in exactly what this session's screenshots do and don't show.

Confirm the AcrPull role assignment

The Identity blade shows the system-assigned identity turned On, but the “Azure role assignments” panel wasn't opened in this session. Confirm (or create) an AcrPull role assignment scoped to registeryr before switching App Service to a container deployment, or the pull will fail with an authorization error.

Resolve the App Service Plan region discrepancy

The Create Web App review screen lists the App Service Plan's region as East US 2, but the live hostname (centralindia-01.azurewebsites.net) and the Container Registry (explicitly Central India) both point to Central India. Worth double-checking the Plan's actual region in Overview so the architecture diagram and any region-specific firewall/network rules are accurate.

Push and deploy the tagged image

The image is already tagged as registeryr.azurecr.io/azure-project2:v1 — the natural next commands are docker push, followed by pointing App Service's Deployment Center at that image (switching Publish from Code to Container).

Capture a local test before pushing

No screenshot shows docker run against the local image. A quick local smoke test (confirm the same page renders on localhost:8000) would catch Dockerfile issues before they reach ACR.

Write down the root cause of the 503 for the README

The incident in Phase 2 of this document — a module-name mismatch between the startup command and webapp.py — is exactly the kind of detail that makes a GitHub README or resume bullet credible. It's worth keeping in the permanent project write-up rather than only in this log.

9. Conclusion

The October 6 session took Project-2 from an empty App Service Plan to a working, monitored Flask deployment — including a real deployment failure, correctly diagnosed through the Kudu SSH console and resolved by fixing a module-name mismatch between the startup command and the deployed file. The same session then opened Phase 2: a Container Registry now exists, a Dockerfile has been written and its image tagged, and a Managed Identity is enabled on the Web App. What remains is mechanical: push the image, confirm the AcrPull role assignment, switch the Web App to a container deployment, and re-run the same verification steps (serial/Kudu access and Application Insights) against the containerized version.